

## PHASE 6

# REVERSE PROXY + LOAD BALANCER

## API Gateway • High-End System Design Interview Notes

### THE BIG IDEA

Reverse Proxy = fronts backend servers. Load Balancer = distributes traffic. API Gateway = central API entry point with API-level policies.

*Front → Distribute → Govern*

## Phase 6 Learning Map

1. Reverse proxy
2. Routing
3. TLS termination
4. Caching and security rules
5. Load balancer
6. Health checks
7. Load-balancing algorithms
8. API gateway
9. Comparison of all three
10. How they work together
11. Failure handling
12. Interview questions

### MEMORY

Do not memorize product names. Understand the primary job of each component.

## 1. Reverse Proxy

A reverse proxy sits in front of backend servers. Clients connect to the proxy rather than directly to the application servers.

- Hides backend topology from clients.
- Routes requests to appropriate backend services.
- Can terminate TLS.
- Can apply security and request policies.
- Can cache suitable responses.
- Can compress responses and manage connections.

Client → Reverse Proxy → Backend Server

### SIMPLE MODEL

The reverse proxy is the public front door to the backend.

## 2. Routing

A reverse proxy can decide where a request should go using the hostname, URL path, headers, or other request information.

```
example.com/users/* → User Service
example.com/orders/* → Order Service
example.com/payments/* → Payment Service
```

- Path-based routing.
- Host-based routing.
- Header-based routing.
- API version routing such as /v1 and /v2.

## 3. TLS Termination

TLS termination means the proxy accepts the client's HTTPS connection and handles the TLS handshake and encryption at that edge.

13. Client connects using HTTPS.
14. Proxy presents the certificate and establishes TLS.
15. Proxy can inspect HTTP-level request information after decryption.
16. Proxy forwards the request to the backend according to the security architecture.

### INTERVIEW POINT

TLS termination centralizes certificate handling. HTTPS can also be maintained between proxy and backend when required.

## 4. ⚡ Reverse Proxy Caching & Security

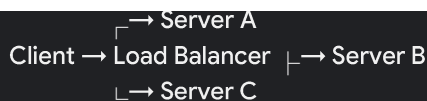
- Cache appropriate public responses.
- Reduce repeated backend work.
- Apply request filtering and security rules.
- Enforce size/method/path restrictions where appropriate.
- Centralize access logging and monitoring.

### CONNECTION TO PHASE 4

Reverse-proxy caching still needs TTL, invalidation, freshness, and stampede considerations.

## 5. ⚖️ Load Balancer

A load balancer distributes incoming requests or connections across multiple backend instances.



- Distributes traffic.
- Supports horizontal scaling.
- Performs health checks.
- Avoids sending normal traffic to unhealthy instances.
- Can improve availability through redundancy.

### PRIMARY JOB

The load balancer's primary responsibility is traffic distribution across backend servers.

## 6. 🩺 Health Checks

A load balancer needs a way to determine whether an instance should receive traffic.

```
GET /health
```

```
200 OK
{"status":"ready"}
```

- A failed health check can remove an instance from the routing pool.
- Readiness and liveness are different concepts.
- Health checks should not create excessive load.

Ask whether the health check proves that the process is alive, ready to serve traffic, or healthy enough to depend on downstream services.

## 7. Load-Balancing Algorithms

| Algorithm            | Meaning                                       | Useful when                                    |
|----------------------|-----------------------------------------------|------------------------------------------------|
| Round Robin          | $A \rightarrow B \rightarrow C \rightarrow A$ | Instances are similar                          |
| Weighted Round Robin | Higher-capacity servers receive more traffic  | Servers differ in capacity                     |
| Least Connections    | Prefer fewer active connections               | Connection load differs                        |
| IP Hash              | Hash determines destination                   | Some session affinity/key locality is required |

## 8. API Gateway

An API gateway is a centralized entry point designed specifically for API traffic and common API policies.

- Routing.
- Authentication and authorization integration.
- Rate limiting and quotas.
- Request/response transformation.
- API versioning.
- Aggregation of multiple backend calls.
- Centralized observability.
- Policy enforcement.



MENTAL MODEL

API Gateway = one API entry point where common API policies live.

## 9. Reverse Proxy vs Load Balancer vs API Gateway

| Component | Primary job | Typical features | Focus |
|-----------|-------------|------------------|-------|
|-----------|-------------|------------------|-------|

## SYSTEM DESIGN • PHASE 6 — REVERSE PROXY + LOAD BALANCER

|               |                         |                                                |                           |
|---------------|-------------------------|------------------------------------------------|---------------------------|
| Reverse Proxy | Fronts backend servers  | Routing, TLS, caching, security                | Backend protection/access |
| Load Balancer | Distributes requests    | Health checks, routing algorithms, failover    | Traffic distribution      |
| API Gateway   | Central API entry point | Auth, rate limits, aggregation, transformation | API policy/control        |

### IMPORTANT

These roles can overlap. A reverse proxy can load-balance, and one product may provide multiple capabilities. Describe the responsibility, not just the product name.

## 10. How They Work Together

```
graph TD
    Client --> CDN[CDN / Edge (optional)]
    CDN --> RP[Reverse Proxy]
    RP --> LB[Load Balancer]
    LB --> HAI[Healthy Application Instances]
    HAI --> CDS[Cache / Database / Services]
```

17. Client sends an HTTPS request.
18. Edge/proxy receives the request.
19. TLS may terminate at the proxy.
20. Routing/security policies are applied.
21. Load balancer selects a healthy backend.
22. Application processes the request.
23. Response travels back through the relevant layers.

### ARCHITECTURE NOTE

You do not always need every layer. Add components because they solve a real requirement, not because the architecture diagram looks more advanced.

## 11. Rate Limiting

Rate limiting controls how much traffic a client, API key, user, or service can generate.

#### SYSTEM DESIGN • PHASE 6 — REVERSE PROXY + LOAD BALANCER

- Protect expensive endpoints.
- Prevent one client from consuming all capacity.
- Set different quotas for different users or API tiers.
- Return HTTP 429 when a configured limit is exceeded.

Client → API Gateway → Rate Limit Check



Allowed / Blocked

#### DESIGN QUESTION

Rate limiting is often placed close to the edge, while sensitive business authorization must still be enforced by backend services.

## 12. Security

- Hide internal backend addresses.
- Terminate or pass through TLS according to the architecture.
- Apply authentication at the appropriate layer.
- Apply authorization for protected resources.
- Filter suspicious or invalid requests.
- Centralize security logging.
- Use defense in depth; backend services should not blindly trust the gateway.

## 13. Failure Handling

- Backend failure → health checks remove the instance.
- Slow backend → timeouts prevent requests waiting forever.
- Repeated dependency failure → circuit breaking/load shedding may help.
- Gateway/proxy failure → deploy redundant instances and health checks.
- Overload → rate limiting, caching, queues, and backpressure can protect dependencies.

#### KEY IDEA

A load balancer does not eliminate failures. It helps the system detect unhealthy instances and route traffic around them.

## 14. Interview Question: Reverse Proxy vs Load Balancer

#### STRONG ANSWER

“A reverse proxy is a front-facing component that hides and protects backend servers and can

provide routing, TLS termination, caching, and security rules. A load balancer's primary job is distributing traffic across multiple healthy backend instances. The roles can overlap, because a reverse proxy can also load-balance."

## 15. Interview Question: What Is an API Gateway?

### STRONG ANSWER

"An API gateway is a centralized entry point for API clients. It routes requests and can provide authentication integration, rate limiting, transformation, versioning, aggregation, and observability. It is useful when multiple services need consistent API-level policies."

## 16. One-Page Cheat Sheet

| Concept           | Remember                                                |
|-------------------|---------------------------------------------------------|
| Reverse Proxy     | Fronts backend servers; routing, TLS, caching, security |
| Load Balancer     | Distributes traffic across backend servers              |
| API Gateway       | Central API entry point; auth, rate limits, aggregation |
| TLS Termination   | Proxy/gateway handles client HTTPS                      |
| Health Check      | Determines whether a backend should receive traffic     |
| Round Robin       | Sequential distribution                                 |
| Least Connections | Prefer fewer active connections                         |
| Rate Limiting     | Controls request volume                                 |
| Core memory       | Front → Distribute → Govern                             |

## 17. Interview Practice

24. What is a reverse proxy?
25. Why hide backend servers behind a reverse proxy?
26. What is TLS termination?
27. How does a load balancer detect an unhealthy server?
28. Explain round robin vs least connections.
29. Why does statelessness help load balancing?

30. Can a reverse proxy also load-balance?

31. What is an API gateway?

32. Why does an API gateway perform rate limiting?

33. What is API aggregation?

34. Do you always need all three components?

35. Design the front door of a high-traffic REST API.

## 18. Mini Design Exercise

You have three API servers and a mobile application with rapidly growing traffic.

36. Expose one stable public endpoint.

37. Terminate HTTPS at the appropriate edge layer.

38. Route requests to the correct service.

39. Distribute traffic across healthy instances.

40. Keep application instances stateless where practical.

41. Apply authentication and rate limits at the gateway when appropriate.

42. Monitor latency, errors, traffic, and backend health.

43. Add instances as demand grows.

### MASTER QUESTION

For every component ask: “What problem does this solve?” If you cannot name the problem, you probably do not need that component.

## PHASE 6 COMPLETE ✓

*Reverse Proxy = Front • Load Balancer = Distribute • API Gateway = Govern*